

Variables & Naming Convention

variables, naming rules, identifiers and literals

Now that you know the grammar of C++ and which names the language has already reserved for itself, the next step is making your own. Every program you write from here on will start by creating variables and giving them names.

Variables

DEFINITION

A Variable is a **named memory location** used to store data.

Simple language — variable ek **container** hai jo data store karta hai. You give the container a name, and later you can look inside it or change what is in it.

MAIN.CPP

```
int age = 20;           // declare and give it a value
age = 21;               // change what is inside
cout << age;           // look inside → 21
```

Notice the shape of that first line — it is the same every time:



Naming Rules

These rules apply to every name you create in C++ — variables, functions, all of it.

Allowed

Letters `a-z` `A-Z`
Digits `0-9`
Underscore `_`

Not Allowed

Starting with a digit
Spaces in the name
Keywords as names

VALID AND INVALID NAMES

```
int age;           // valid
int _count;        // valid
int total2;        // valid - digit is fine, just not first

int 2total;        // invalid - starts with a digit
int my age;        // invalid - space in the name
int float;         // invalid - float is a keyword
```



Some compilers such as GCC and Clang also accept `$` in names — but that is a non-standard extension. Stick to letters, digits and `_` so your code works everywhere.

NAMING STYLES

USED FOR	STYLE	EXAMPLE
Class	PascalCase	StudentRecord

USED FOR	STYLE	EXAMPLE
Variable / Function	camelCase or snake_case	totalMarks · total_marks

Identifiers & Literals

IDENTIFIER

An Identifier is the **name given** to variables, functions, classes, arrays and so on.

LITERAL

A Literal is a **fixed value written directly** in the program.

The easiest way to tell them apart is to look at one line of code:

IDENTIFIER VS LITERAL

```
int marks = 95;
//  ^^^^^  ^^
//  identifier (the name you made)
//           literal (the fixed value)
```

LITERAL TYPE	EXAMPLE
Integer literal	10 · -5 · 0
Floating literal	3.14 · 9.8

LITERAL TYPE	EXAMPLE
Character literal	'A' · 'z' – single quotes
String literal	"Hello" – double quotes
Boolean literal	true · false



'A' and "A" are not the same thing. Single quotes make a **character**; double quotes make a **string**. Mixing them up is a very common early mistake.